

Python学習 3日目午後

超サポ
愉快カンパニー

アシスト

アジェンダ

アジェンダ

1 例外処理について
try-except構文でエラーに対処する

2 Pythonライブラリについて
import文で便利な機能を使う

3 デコレーターについて
@記号で関数に機能を追加する

4 クラスについて
データと処理をひとまとめにする

5 今フェーズのまとめ
学習内容の振り返り

6 アプリ開発
クッキングアプリケーションを開発

フェーズ目標成果物

print() から Class までの
学習内容を使用した

**クッキング
アプリケーション**

QAシート

- 気になったことについては気軽にQAシートに記入してください
 - https://docs.google.com/spreadsheets/d/1MxqWSRFn6r3BCzAa17TtrWa_HoRhxp0aUoN6Su5KDb0/edit?usp=drive_link

例外処理について

日常生活における例外処理

プログラムを実行していると、予期しないエラーが発生することがある。例外処理は、このようなエラーに対処する仕組みである。



ATMの引き出し

お金が足りない場合
→ 取引を中止して通知



ファイルを開く

ファイルが存在しない場合
→ エラーメッセージを表示



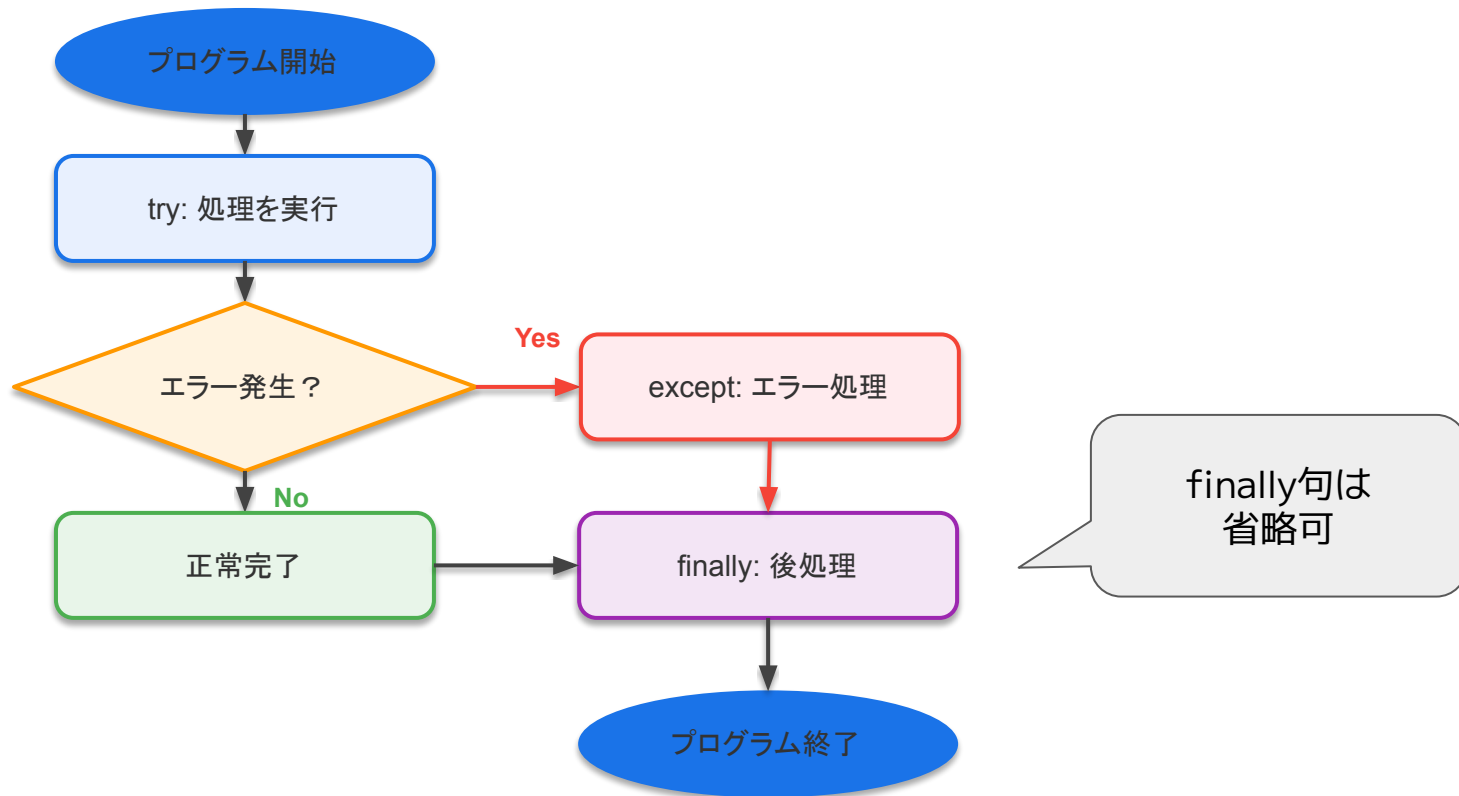
ネットワーク接続

接続が切れた場合
→ 再接続を試みる

SnowflakeやDataSpiderなどでは日常的に使われている

例外処理の流れ

エラーが発生したときのプログラムの動きを理解しましょう



try-except の基本構文

以下のコードを書いてみましょう

try:

```
# エラーが起きる可能性のある処理  
result = 10 / 0
```

except ZeroDivisionError:

```
# エラーが起きたときの処理  
print("ゼロで割ることはできません")
```

ポイント

- try: エラーが起きる可能性のあるコードを書く
- except: エラーが起きたときの処理を書く
- 例外の種類を指定できる (ZeroDivisionError等)
- コロン(:)は必須

① try:

エラーが起きるかもしれない
コードを実行する

② except:

エラーが起きたら、
ここの処理を実行する

③ コロン(:)

try: と except: の末尾に
コロンは必須

実行例: ファイル読み込み

try:

```
file = open("data.txt", "r")  
content = file.read()  
file.close()
```

except FileNotFoundError:

```
print("ファイルが見つかりません")
```

ポイント

- ファイルが存在しない場合、FileNotFoundErrorが発生する
- exceptで例外をキャッチし、エラーメッセージを表示する
- プログラムがクラッシュせずに続行できる

※ 実際の開発では with 文を使ったファイル操作が推奨されます

複数の例外処理

try:

```
num = int(input("数字を入力:"))  
result = 100 / num  
print(result)
```

except ValueError:

```
print("数字を入力してください")
```

except ZeroDivisionError:

```
print("ゼロでは割れません")
```

ポイント

- 複数のexceptを使って、異なるエラーに対処できる
- 上から順に判定される
- 最初にマッチしたexceptが実行される

※ 上から順番に判定され、最初に当てはまったものだけが実行される

finally: 必ず実行される処理

try:

```
file = open("data.txt", "r")  
content = file.read()
```

except FileNotFoundError:

```
print("ファイルが見つかりません")
```

finally:

```
# エラーの有無にかかわらず実行される  
print("処理を終了します")
```

ポイント

- finallyブロックは、エラーの有無にかかわらず必ず実行される
- ファイルを閉じる、接続を切断するなどの後処理に使用
- try-except-finally の順で記述する

演習: 入力値の例外処理

問題

ユーザーから数値を入力してもらい、その数値を2倍にして表示するプログラムを作成してください。

ただし、数値以外(文字列など)が入力された場合は「正しい数値を入力してください」と表示する。

ヒント

`int()` でエラーが発生する場合を考える
`try-except` を使って `ValueError` をキャッチする

まとめ:例外処理

構文	用途	注意点
try-except	エラー時の処理	コロン必須
try-except-finally	必ず実行する処理	インデント必須
複数のexcept	複数種類のエラー対応	上から順に判定

- ✓ try-exceptでエラーに対処する
- ✓ exceptで例外の種類を指定できる
- ✓ finallyは必ず実行される

Pythonライブラリについて

ライブラリとは

ライブラリは、便利な機能をまとめたプログラムの集まりである。
自分でゼロから作らなくても、ライブラリを使えば複雑な処理が簡単にできる。



スマホアプリ

後からアプリ一覧に追加することができる
→ 機能を拡張することができる



math

数学の関数
平方根、三角関数、
円周率など



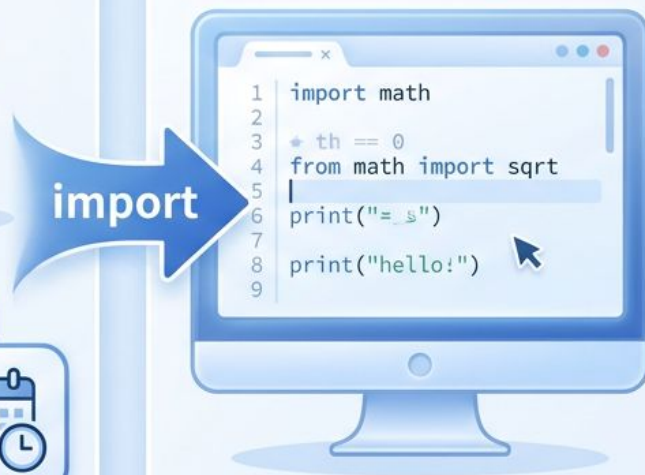
random

乱数の生成
ランダムな整数、
リストからの選択

1) ライブラリ (Library)



2) インポート (Import)



3) インポートの書き方 (Import Styles)



`import math`

ライブラリ全体をインポート



`from math import sqrt`

特定の関数のみをインポート



`import datetime as dt`

別名を付けてインポート

ライブラリのインポート

import文でライブラリを読み込みます

```
import math
```

```
# mathライブラリの関数を使用
```

```
result = math.sqrt(16)
```

```
print(result) # 4.0
```

ポイント

- import文でライブラリを読み込む
- ライブラリ名.関数名() で使用する
- Pythonに標準で含まれるパッケージを「標準パッケージ」と呼ぶ

① import ライブラリ名

パッケージ全体を読み込む

② ライブラリ名.関数名()

使いたい機能呼び出す

特定の機能 (モジュール) だけインポート

```
from math import sqrt, pi
```

```
# math.を付けずに使える
```

```
result = sqrt(25)
```

```
print(pi) # 3.141592...
```

ポイント

- from ライブラリ名 import 関数名
- ライブラリ名を省略して使える
- 必要な機能だけを読み込むことができる

実行例: randomライブラリ

```
import random
```

```
# 1から10のランダムな整数
```

```
num = random.randint(1, 10)
```

```
print(num)
```

```
# リストからランダムに選択
```

```
fruits = ["りんご", "バナナ", "みかん"]
```

```
choice = random.choice(fruits)
```

```
print(choice)
```

ポイント

- randomライブラリは乱数生成に使用する
- randint(): 指定範囲の整数を生成
- choice(): リストからランダムに選択
- shuffle(): リストをシャッフル

実行例: datetimeライブラリ

import datetime

現在の日時を取得

```
now = datetime.datetime.now()  
print(now)
```

日付のみ取得

```
today = datetime.date.today()  
print(today)
```

ポイント

- datetimeライブラリは日付・時刻の操作に使用する
- now(): 現在の日時を取得
- today(): 今日の日付を取得
- 日付の計算やフォーマットも可能

エイリアスを付けてインポート

```
import datetime as dt
```

```
# dt という短い名前で使える  
now = dt.datetime.now()  
print(now)
```

ポイント

- as で別名(エイリアス)を付けられる
- 長いライブラリ名を短縮できる
- 例: numpy → np, pandas → pd

書き方	使い方	例
import math	math.sqrt(16)	ライブラリ全体を読み込む
from math import sqrt	sqrt(16)	特定の機能のみ読み込む
import datetime as dt	dt.datetime.now()	別名を付けて読み込む

演習:ライブラリの活用

問題

randomライブラリを使って、
1から100までのランダムな整数を5個表示するプログラムを作成する。

ヒント

for文と random.randint() を組み合わせる
range(5) で5回繰り返す

まとめ:ライブラリ

構文	使い方	主なライブラリ
import	ライブラリ全体を読み込む	math, os
from import	特定の機能のみ読み込む	random, datetime
import as	別名を付けて読み込む	numpy, pandas

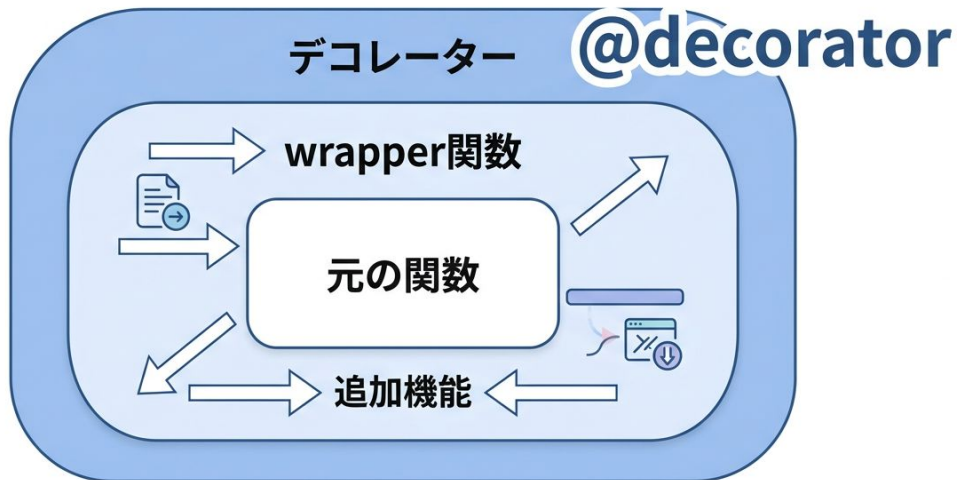
- ✓ ライブラリで複雑な処理が簡単にできる
- ✓ import文で読み込む
- ✓ 用途に応じてインポート方法を使い分ける

デコレーターについて

デコレーターとは

デコレーターは、関数に機能を追加するための仕組みである。既存の関数を変更せずに、新しい機能を付け加えることができる。

Python デコレーターの仕組み

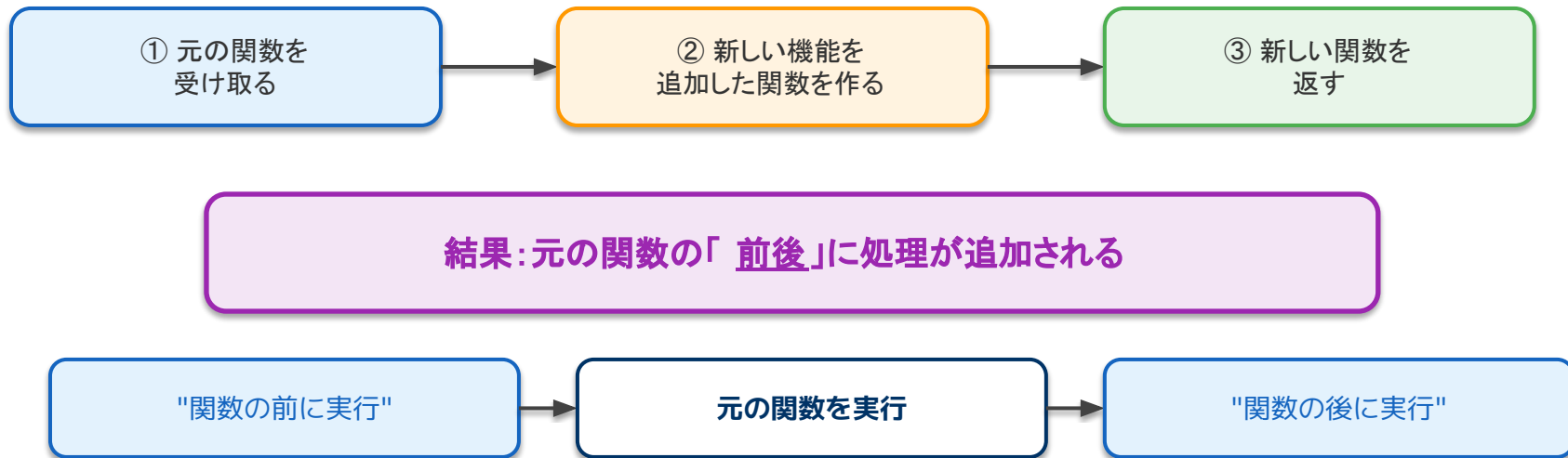


日常の例え

料理をする
(中身 = 元の関数)

経過時間を計測する
(装飾 = デコレーター)

デコレーターの仕組み



デコレーターの基本

以下のコードを書いてみましょう

```
def my_decorator(func):  
    def wrapper():  
        print("関数の前に実行")  
        func()  
        print("関数の後に実行")  
    return wrapper
```

```
@my_decorator  
def say_hello():  
    print("Hello!")  
  
say_hello()
```

ポイント

- @記号でデコレーターを適用する
- 関数の前後に処理を追加できる
- 元の関数を変更しない
- wrapperが新しい関数として返される

実行結果:

```
関数の前に実行  
Hello!  
関数の後に実行
```

実行例: 実行時間を測定するデコレーター

```
import time
```

```
def timer(func):
```

```
    def wrapper():
```

```
        start = time.time()
```

```
        func()
```

```
        end = time.time()
```

```
        print(f"実行時間: {end - start}秒")
```

```
    return wrapper
```

```
@timer
```

```
def slow_function():
```

```
    time.sleep(2)
```

```
    print("処理完了")
```

```
slow_function()
```

ポイント

- 関数の実行時間を自動的に測定する
- 元の関数を変更せずに、測定機能を追加できる
- 複数の関数に同じデコレーターを適用可能

引数を持つ関数のデコレーター

```
def repeat(num):  
    def decorator(func):  
        def wrapper(*args, **kwargs):  
            for i in range(num):  
                func(*args, **kwargs)  
        return wrapper  
    return decorator
```

```
@repeat(3)  
def greet(name):  
    print(f"こんにちは、{name}さん")  
  
greet("太郎")
```

ポイント

- *args, **kwargsで任意の引数に対応
- デコレーターに引数を渡せる
- 3重のネストになる点に注意

演習: ログ出力デコレーター

問題

関数が呼び出されたときに

「〇〇関数が実行されました」と表示するデコレーターを作成してください。

ヒント

`func.__name__` で関数名を取得できる
wrapper関数の中で `print()` を使う

まとめ:デコレーター

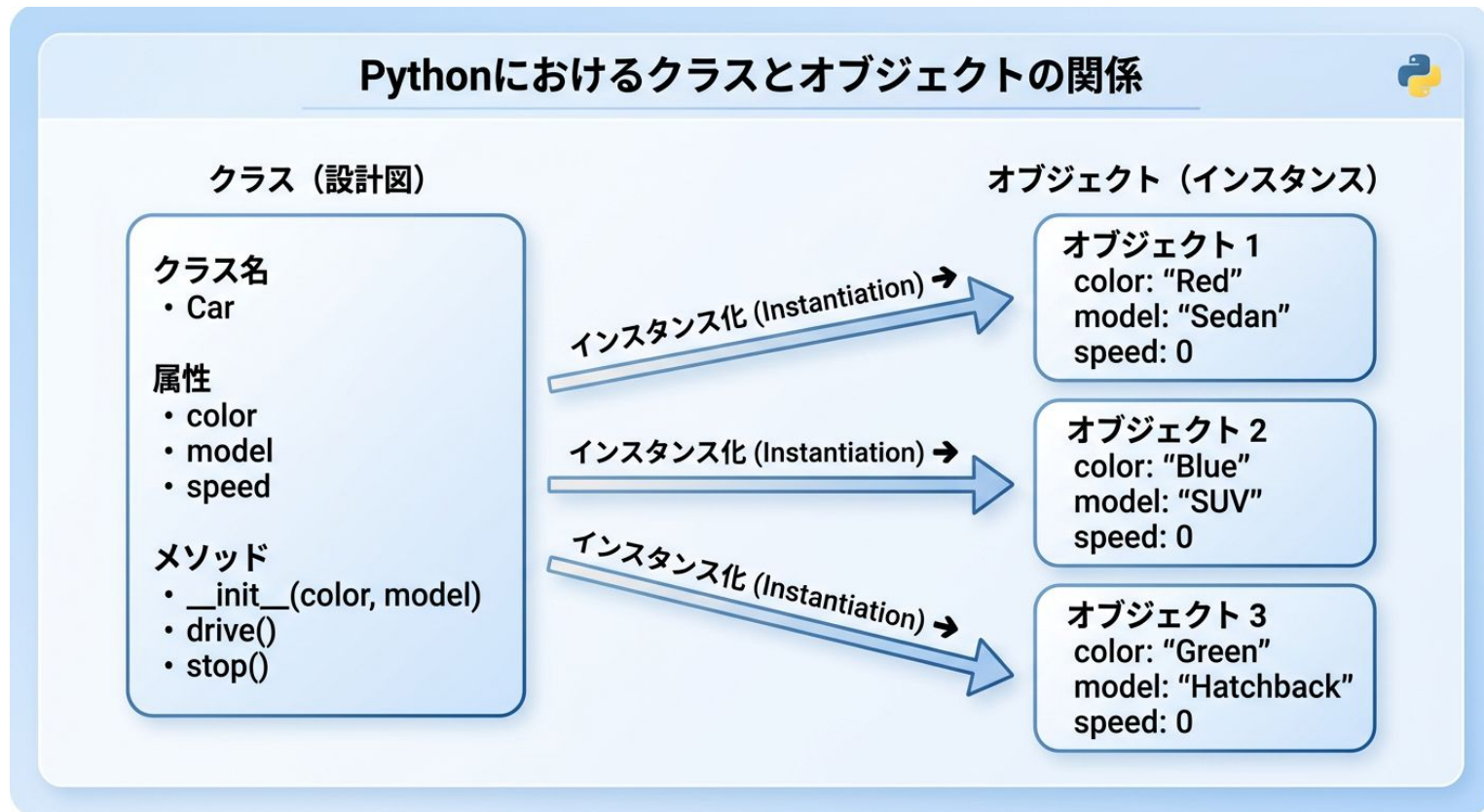
構文	用途	活用例
@デコレーター	関数の拡張	ログ出力
def wrapper	内部関数	実行時間測定
*args, **kwargs	任意の引数	認証チェック

- ✓ デコレーターで関数に機能を追加する
- ✓ @記号で適用する
- ✓ 元の関数を変更しない

クラスについて

クラスとは

クラスは、データと処理をひとまとめにする仕組みである。設計図のようなもので、そこから具体的なオブジェクトを作る。



クラスとオブジェクトの関係

① クラス(設計図)
を定義

② オブジェクト(実体)
を作成

③ メソッドを
実行

```
class Dog:          # ① クラスを定義
    # 初期化メソッド(オブジェクトを作るときに呼ばれる)
    def __init__(self, name, age):
        self.name = name # 属性:名前
        self.age = age   # 属性:年齢

    # メソッド:吠える
    def bark(self):
        print(f"{self.name}がワン!と吠えた")
```

```
my_dog = Dog("ポチ", 3) # ② オブジェクトを作成
my_dog.bark()           # ③ メソッドを実行 → "ポチがワン!と吠えた"
```

クラス図 (イメージ)

Dog
name: str age: int
bark()

クラスの基本構文

以下のコードを書いてみましょう

```
class Dog:
```

```
    # 初期化メソッド
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    # メソッド (関数)
```

```
    def bark(self):
```

```
        print(f"{self.name}がワンワン！")
```

```
# オブジェクトの作成
```

```
my_dog = Dog("ポチ", 3)
```

```
my_dog.bark()
```

ポイント

- **class** クラス名: でクラスを定義
- **__init__**: 初期化メソッド(コンストラクタ)
- **self**: オブジェクト自身を指す(必須)
- メソッドの第1引数は常に **self**

- ① **class**: クラスを定義する
- ② **__init__**: オブジェクト作成時に実行
- ③ **self**: オブジェクト自身を表す(必須)

実行例: Personクラス

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    def introduce(self):
```

```
        print(f"私は{self.name}、{self.age}歳  
です")
```

```
person1 = Person("田中", 25)
```

```
person2 = Person("佐藤", 30)
```

```
person1.introduce()
```

```
person2.introduce()
```

ポイント

- 同じクラスから複数のオブジェクトを作成できる
- 各オブジェクトは独立したデータを持つ
- メソッドは全オブジェクトで共通

クラス変数とインスタンス変数

```
class Counter:
```

```
    # クラス変数 (全オブジェクトで共有)
```

```
    total = 0
```

```
    def __init__(self):
```

```
        # インスタンス変数 (オブジェクトごとに独立)
```

```
        Counter.total += 1
```

```
        self.id = Counter.total
```

```
c1 = Counter()
```

```
c2 = Counter()
```

```
print(Counter.total) # 2
```

ポイント

- クラス変数: すべてのオブジェクトで共有
- インスタンス変数: オブジェクトごとに独立
- クラス変数はクラス名.変数名でアクセス

クラス変数

total = 2
(全体で共有)

インスタンス変数

c1.id=1, c2.id=2
(個別に独立)

継承: クラスを拡張する

```
class Animal:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def speak(self):
```

```
        print("何か鳴く")
```

```
class Cat(Animal):
```

```
    def speak(self):
```

```
        print(f"{self.name}がニャー")
```

```
cat = Cat("タマ")
```

```
cat.speak()
```

ポイント

- class 子クラス(親クラス): で継承
- 親クラスのメソッドを上書き(オーバーライド)できる
- 親クラスの機能を再利用できる

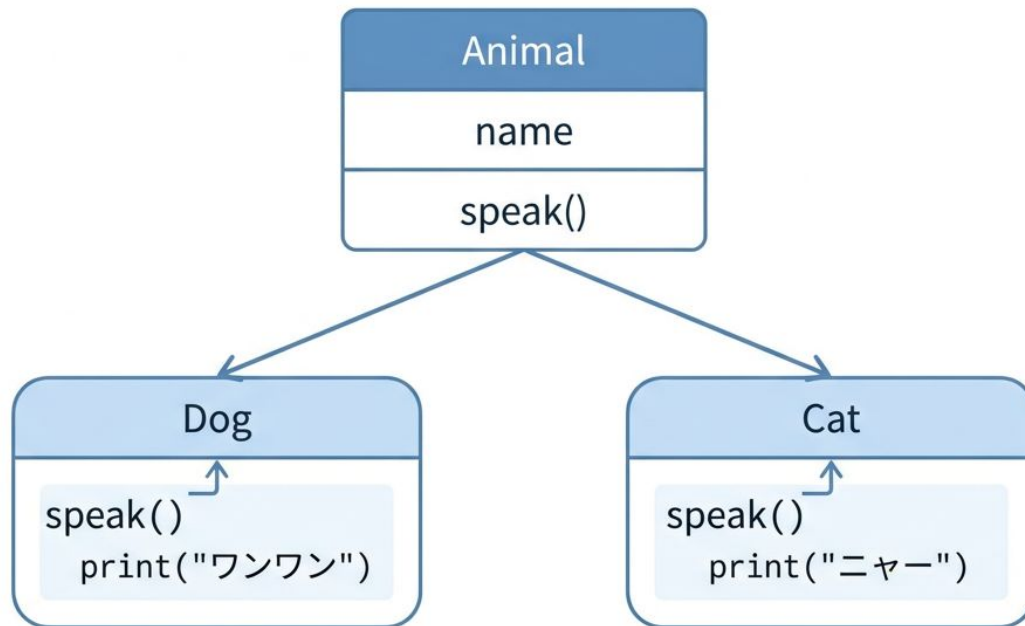
Animal(親クラス)

継承

Cat(子クラス)

継承のイメージ

Python Class Inheritance and Method Overriding



演習: Bookクラスの作成

問題

書籍を表すBookクラスを作成してください。

title(タイトル)、author(著者)、price(価格)の属性を持ち、

show_info() メソッドで情報を表示する。

ヒント

`__init__` で属性を初期化する

`show_info()` で `print()` を使って表示する

f文字列を使うと便利

まとめ: クラス

構文	意味	応用
class	クラス定義	継承
__init__	初期化	メソッド
self	自身を指す	属性アクセス

- ✓ クラスはデータと処理をまとめる
- ✓ __init__で初期化する
- ✓ selfでオブジェクト自身を指す
- ✓ 継承で機能を拡張できる

今フェーズのまとめ

学習内容の振り返り

トピック	キーワード	用途
例外処理	try-except	エラー対処
ライブラリ	import	便利な機能
デコレーター	@デコレーター	関数拡張
クラス	class	データと処理

- ✓ 例外処理でプログラムを安定させる
- ✓ ライブラリで開発を効率化する
- ✓ デコレーターで機能を追加する
- ✓ クラスでコードを整理する

これらの知識を組み合わせる

例外処理 + ライブラリ → 安全なファイル操作

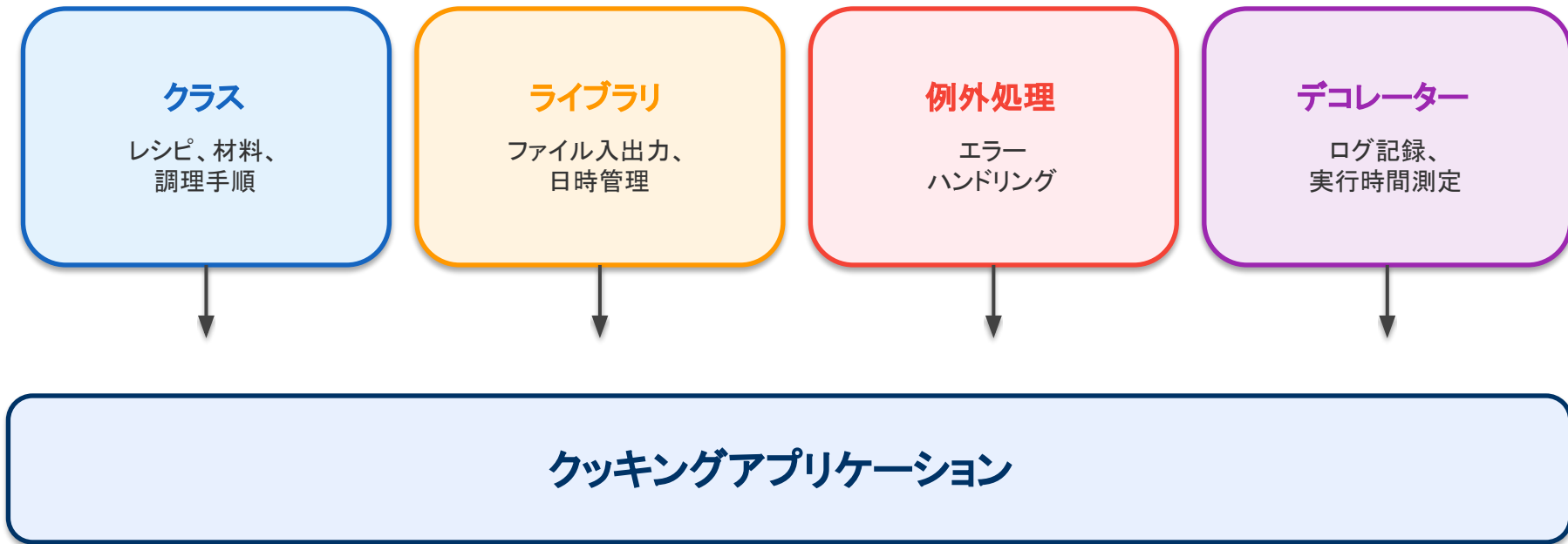
クラス + デコレーター → 拡張可能な設計

すべてを組み合わせる → 実用的なアプリケーション

学習内容を踏まえてのアプリ開発

目標成果物

print()からClassまでの学習内容を使用したクッキングアプリケーションを開発する。



料理アプリケーションのPython構成図

クラス (Class)

ビジネスロジックとデータの構造



レシピ
(Recipe)



食材
(Ingredient)



調理手順
(CookingStep)

ライブラリ (Library)

recipe_data.json



ファイル入出力
(File I/O)



日時管理
(Datetime)

例外処理



エラー処理
(Error Shield)

デコレーター



ログ出力
(Log Output)



処理時間計測
(Timer)

実装例: レシピクラス

```
import datetime
```

```
class Recipe:
```

```
    def __init__(self, name, ingredients):
```

```
        self.name = name
```

```
        self.ingredients = ingredients
```

```
        self.created_at = datetime.datetime.now()
```

```
    def show_recipe(self):
```

```
        print(f"レシピ: {self.name}")
```

```
        print("材料:")
```

```
        for ing in self.ingredients:
```

```
            print(f" - {ing}")
```

```
recipe = Recipe("カレー", ["肉", "野菜", "カレー  
ルー"])
```

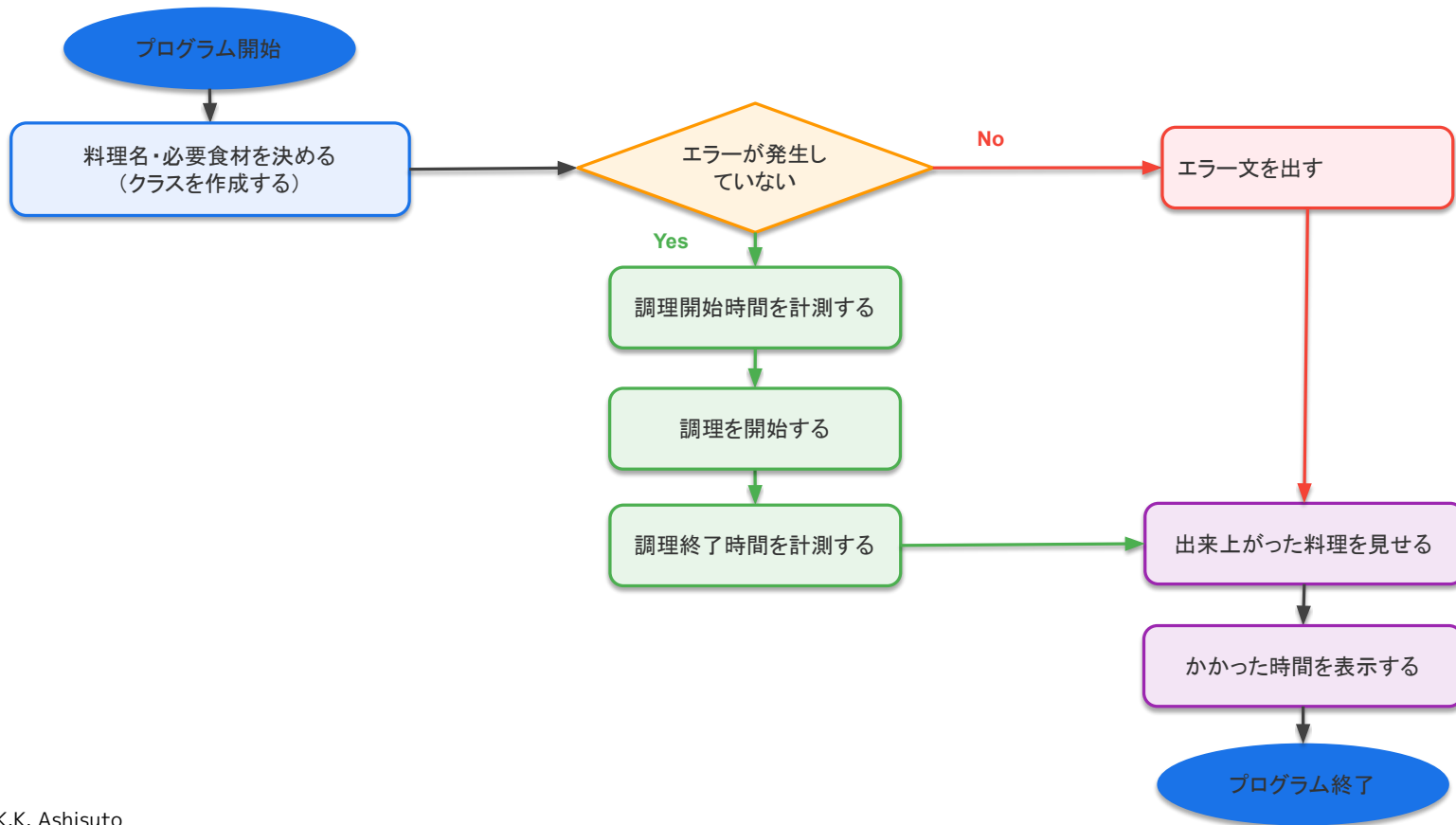
```
recipe.show_recipe()
```

ポイント

- クラスでレシピを管理
- ライブラリ(datetime)で作成日時を記録
- メソッドでレシピ情報を表示
- リストで材料を管理

アプリを開発するときの進め方

アプリがどんな動きをするのかのフローチャートを作成しましょう



次のステップ

1

学んだ知識を実践する

実際にコードを書いて動かしてみる

2

エラーに対処しながら開発を進める

try-exceptを活用する

3

コードを整理して読みやすくする

クラスやデコレーターで構造化

4

継続的に学習を続ける

新しいライブラリや技術を習得

QA回答
